

Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)

Department of CIS, Room F-1030-I

Office hours: Mon 2pm-5pm (in-person and Zoom)

Office phone: 212-220-1490

Borough of Manhattan Community College/CUNY

CSC331 Fall 2025

Data Structures

Week 3:

Linked list of sorted integers
Unique & sorted integers implementation
Input command string parsing

Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- Program 1 due date
 - 1159pm Fri 9-19-2025
 - late submission receives no credit
- No class Mon 9-22-2025 thru Wed 9-24-2025, Wed 10-1-2025, Thu 10-2-2025
- Exam 1
 - Section 501 Tue 9-30-2025 – Zoom 630pm-730pm
 - Section 172 Wed 10-8-2025 – In person 630pm-730pm
 - Closed book, closed notes, no devices
 - Section 500 Wed 10-8-2025 – In person 1010am-1110am
 - Closed book, closed notes, no devices
- Program 2 due date
 - 1159pm Fri 10-10-2025
 - late submission receives no credit
- Posted on Brightspace
 - Week 3 lecture slides, sample c++ string operations (chap7h.cpp, chap7k.cpp), buildListForward function implementation - in ascending order chap5c.cpp

Linked lists

```
// chap5a.cpp
// kbyron@bmcc.cuny.edu
// 2-14-2019
// create and display linked list (from Malik chapter 5)
#include <iostream>
using namespace std;
struct nodeType{
    int info;
    nodeType *link;
};
nodeType* buildListForward(); // function prototype
int main(){
    nodeType *current,*head;
    head=buildListForward();
    current=head;
    while(current!=NULL){
        cout << current->info << " ";
        current=current->link;
    }
    cout << endl;
    return 0;
}
nodeType* buildListForward() // function definition
```

Linked lists

```
nodeType* buildListForward() // function definition
{
    nodeType *first, *newNode, *last;
    int num;
    cout << "Enter a list of integers ending with -999." << endl;
    cin >> num;
    first = NULL;
    last = NULL;
    while (num != -999)
    {
        newNode = new nodeType;
        newNode->info = num;
        newNode->link = NULL;
        if (first == NULL)
        {
            first = newNode;
            last = newNode;
        }
        else
        {
            last->link = newNode;
            last = newNode;
        }
        cin >> num;
    } //end while
    return first;
} //end buildListForward
```

Linked lists

- Sorted integer implementation

```
delltak-2022 fy23 >: ./chap5c
Enter a list of integers ending with -999.
22
successful insertion
44
successful insertion
66
successful insertion
33
successful insertion
-2
successful insertion
-999
List: -2 22 33 44 66
```

Linked lists

- Sorted integer implementation

```
// chap5c.cpp
// kbyron@bmcc.cuny.edu
// 9-15-2022 revised 2-10-2025
// create and display an ordered linked list of integers
#include <iostream>
using namespace std;
struct nodeType{
    int info;
    nodeType *link;
};
// -----
nodeType* buildListFwdOrdered();           // function prototype
void insertNum(nodeType* &head,int num);   // function prototype
void showList(nodeType *head);             // function prototype
// -----
int main(){
    nodeType *head=buildListFwdOrdered();
    showList(head);
}
```

Linked lists

- Sorted integer implementation

```
nodeType* buildListFwdOrdered() { // function definition
    nodeType *first;
    int num;
    cout << "Enter a list of integers ending with -999." << endl;
    cin >> num;
    first = NULL;
    while (num != -999) {
        insertNum(first, num);
        cout << "successful insertion" << endl;
        cin >> num;
    }
    return first;
}
```

Linked lists - Sorted integer implementation

```
void insertNum(nodeType* &first, int num){
    nodeType *current, *newNode, *prior;
    if(first == NULL){ // insert into empty list
        first = new nodeType;
        first->info = num;
        first->link = NULL;
    }
    else{
        if(first->info > num){ // insert at top of list
            newNode = new nodeType;
            newNode->info = num;
            newNode->link = first;
            first=newNode;
        }
        else{
            int done = 0;
            current = first;
            while((current != NULL) && (!done)){
                if(current->info > num){ // insert between 2 nodes
                    newNode = new nodeType;
                    newNode->info = num;
                    newNode->link = current;
                    prior->link = newNode;
                    done = 1;
                }
                else{
                    prior = current;
                    current = current->link;
                }
            }
            if(!done){ // insert at end of list
                newNode = new nodeType;
                newNode->info = num;
                newNode->link = current;
                prior->link = newNode;
            }
        }
    }
}
```

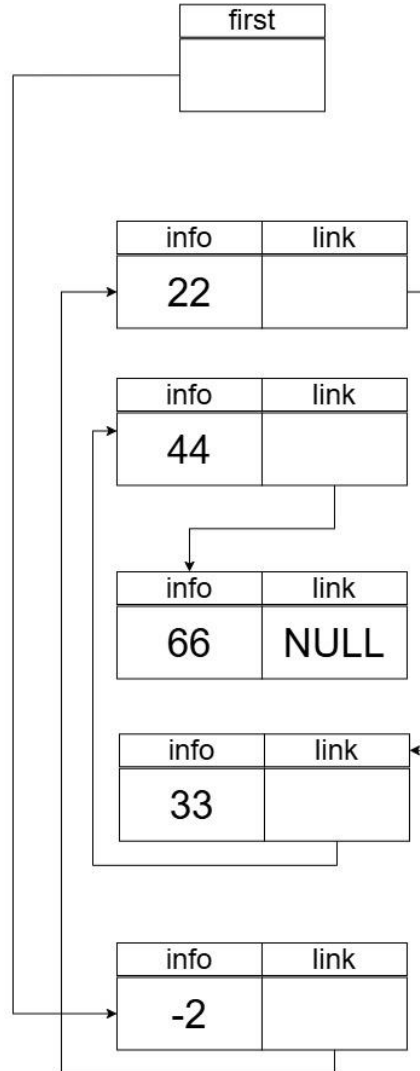
Linked lists

- Sorted integer implementation

```
void showList(nodeType *head){ // function definition
    nodeType *current;
    current = head;
    cout << "List: ";
    while(current!=NULL) {
        cout << current->info << " ";
        current=current->link;
    }
    cout << endl;
}
```

Linked lists

- Linked list produced by chap5c.cpp



Linked lists

- Sample C++ string operations (see chap7k.cpp)``

```
#include <string>
using namespace std;
int main () {
    string name, t;
    int age, namelen, pos1, pos2;
    char cmd=' ';
    while (cmd!='Q'){
        cout << "enter transaction ... ";
        getline (cin, t);
        cmd=t[0];
        cout << "cmd = " << cmd << endl;
        if (cmd=='A'){
            pos1=1;
            pos2=t.find(' ',pos1+1);
            namelen=pos2-pos1-1;
            name=t.substr(pos1+1,namelen);
            age=stoi(t.substr(pos2+1));
            cout<<name<<" "<<age<<endl;
        }
        if (cmd=='D'){
            pos1=1;
            pos2=t.find(' ',pos1+1);
            namelen=pos2-pos1-1;
            name=t.substr(pos1+1,namelen);
            cout<<name<<endl;
        }
    }
}
```

Linked lists

- Sample C++ string operations

```
// chap7h.cpp
// kbyron@bmcc.cuny.edu
// last revised 2-24-2021
// C program to illustrate some C++ i-o and string class functions
// cloned from chap7g.cpp
#include <iostream>
#include <string>
using namespace std;
int main () {
    string name1="yes", name2="yes";
    while(!(name1=="no"&&name2=="no")){
        cout << "enter 2 names separated by space (enter 'no no' when done) ... ";
        cin >> name1 >> name2;
        int result = name1.compare(name2);
        cout << " name1 = " << name1 << " name2 = " << name2 << " result = " << result << endl;
        if(result==0)
            cout << name1 << " = " << name2 << endl;
        else
            if(result<0)
                cout << name1 << " < " << name2 << endl;
            else
                cout << name1 << " > " << name2 << endl;
        if(name1>name2)
            cout << "name1 is greater" << endl;
        else
            if(name1<name2)
                cout << "name1 is less" << endl;
            else
                if(name1==name2)
                    cout << "name1 is equal" << endl;
    }
}
```

Linked lists

- Sample C++ string operations

```
}  
string E ("23+5*8-");  
cout << "enter a post-fix expression (enter 'quit' when done) ... ";  
cin >> E;  
while(E.compare("quit") != 0){  
    for (int i=0; i<E.length(); i+=1){  
        cout << E[i] << " ";  
        char x = E[i];  
        if((x >= '0')&&(x <='9')){  
            cout << " digit \n";  
        }  
        else{  
            cout << " operator \n";  
        }  
    }  
    cout << "enter a post-fix expression (enter 'quit' when done) ... ";  
    cin >> E;  
}  
cout << "good-bye." << endl;  
return 0;  
}
```

String data vs numeric data

- ASCII (American standard code for information interchange) conversion chart

Decimal - Binary - Octal - Hex – ASCII
Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	"	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	\$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	'	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

Linked lists

- Sample C++ string operations

Function Name and Parameters	Parameter(s) Type	Function Return Value
<code>strcmp(str1, str2)</code>	<code>str1</code> and <code>str2</code> are null terminated char arrays	The returned value is as follows: - An int value < 0, if <code>str1 < str2</code> - An int value 0, if <code>str1 = str2</code> - An int value > 0, if <code>str1 > str2</code>
<code>getline(istreamVar, str)</code>	<code>istreamVar</code> is an input stream variable (of type <code>istream</code> or <code>ifstream</code>). <code>str</code> is a <code>string</code> object (variable).	Characters until the newline character are input from <code>istreamVar</code> and stored in <code>str</code> . (The newline character is read but not stored into <code>str</code> .) The value returned by this function is usually ignored.

Linked lists

- Sample C++ string operations

Function Name and Parameters	Parameter(s) Type	Function Return Value
<code>str.find(strExp)</code>	<code>str</code> is a string object and <code>strExp</code> is a string expression evaluating to a string. The string expression, <code>strExp</code> , can also be a character.	The <code>find</code> function searches <code>str</code> to find the first occurrence of the string or the character specified by <code>strExp</code> . If the search is successful, the function <code>find</code> returns the position in <code>str</code> where the match begins. If the search is unsuccessful, the function returns the special value <code>string::npos</code> .
<code>str.substr(pos, len)</code>	Two unsigned integers, <code>pos</code> and <code>len</code> . <code>pos</code> , represent the starting position (of the substring in <code>str</code>), and <code>len</code> represents the length (of the substring). The value of <code>pos</code> must be less than <code>str.length()</code> .	A temporary string object that holds a substring of <code>str</code> starting at <code>pos</code> . The length of the substring is, at most, <code>len</code> characters. If <code>len</code> is too large, it means “to the end” of the string in <code>str</code> .

Assignment

- Read Malik chapter 5 - linked lists
- Read Malik Appendix E – useful functions/methods: `getline`, `str.find()`, `str.cmp()`, `str.substr()`
- Download and test cpp samples on Blackboard